

Architecture logicielle & systèmes distribués

[Accueil](#)[Introduction](#)[Jour 1](#)[Jour 2](#)[Jour 3](#)[Jour 4](#)[Jour 5](#)[Jour 6](#)[TP1](#)[TP2](#)[TP3](#)[Exercices](#)[Corrigés](#)[Glossaire](#)[Live Coding](#)[Dossier Architecture](#)[FAQ](#)[Support HTML modulaire](#)[Master 2](#)[6 jours](#)[3 TP progressifs](#)[Java / Spring Boot](#)

TP2 — Passage en distribué

Objectifs pédagogiques

Faire évoluer le projet fil rouge en architecture distribuée simple et contrôlée.

Temps estimé

≈ 6h incluant démos et quiz

Prérequis

TP1 terminé + Jour 3 et 4

Table des matières

- **Objectifs**
- Énoncé
- Étapes
- Structure
- Code
- Pièges
- Évaluation
- Corrigé

Objectifs pédagogiques

- Découper progressivement le projet du TP1.
- Introduire communication inter-services et API Gateway.
- Observer les impacts du distribué : erreurs, temps réseau, contrats.

Énoncé

À partir du monolithe du TP1, extraire :

- `catalog-service`
- `loan-service`
- `api-gateway`

Le `loan-service` doit appeler le `catalog-service` pour vérifier la disponibilité d'un livre avant de créer le prêt.

Étapes détaillées

1. Identifier les frontières métier.
2. Créer un contrat HTTP interne simple.
3. Déplacer les données par service.
4. Mettre en place Gateway.
5. Tester les cas d'erreur : service indisponible, réponse lente, contrat cassé.

Structure de projet attendue

```
distributed-library/  
├─ api-gateway  
├─ catalog-service  
└─ loan-service
```

Extraits de code commentés

```
@FeignClient(name = "catalog-service", url = "${catalog.url}")  
public interface CatalogClient {  
    @GetMapping("/internal/books/{bookId}/availability")  
    AvailabilityResponse getAvailability(@PathVariable UUID bookId);  
}
```

```
public LoanResponse createLoan(CreateLoanRequest request) {  
    AvailabilityResponse availability = catalogClient.getAvailability(request.bookId());  
    if (!availability.available()) {  
        throw new BusinessException("Livre non disponible");  
    }  
    // créer le prêt localement  
}
```

Pièges fréquents

- Conserver une base partagée entre services.
- Multiplier les appels réseau dans une boucle.
- Ne pas documenter le contrat interne.
- Éparpiller les URLs de services en dur.

Critères d'évaluation

- Découpage justifié.
- Contrats inter-services clairs.
- Gateway cohérente.
- Gestion minimale des erreurs.
- Compréhension explicite des nouveaux risques introduits.

Corrigé détaillé

Le bon corrigé montre que le passage au distribué n'est pas "gratuit" : on gagne en séparation mais on paie en complexité réseau, debug et déploiement.

Ce que l'étudiant doit avoir compris

Couper une application en morceaux n'est utile que si chaque morceau a un sens clair et des interactions maîtrisées.

[← TP1](#)[TP3 →](#)